HTML BASICS & STRUCTURE

Complete Beginner-Friendly Notes

12 topics • Concepts • Examples • Best practices • Practice questions

What you will learn

- How an HTML document is structured and how tags, elements, and attributes work.
- How to format text, create links, add images and media, and organize information.
- How to build forms, use semantic HTML5, and improve SEO and accessibility.
- How entities, responsive basics, and clean coding practices make pages more reliable.

Made for learners starting HTML from zero. Examples are designed to be copied into an .html file and tested in a browser.

How to use these notes

HTML (HyperText Markup Language) describes the structure and meaning of web content. It is not a programming language: HTML marks up content, CSS styles it, and JavaScript adds behavior.

- Create a folder, then create a file named index.html.
- Write an example, save the file, and open it in a browser.
- Change one thing at a time and observe the result.
- Use a code editor such as VS Code; keep indentation consistent.

A minimal HTML page

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, world!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Remember: The head contains page information; the body contains the visible page content.

01. HTML Basics & Structure

Core terms

- Tag: markup written inside angle brackets, such as .
- Element: usually the opening tag, content, and closing tag together, such as Hello .
- Attribute: extra information written in an opening tag, such as href or class.
- Void element: an element that does not wrap content, such as <img>,
, and <input>.

Document structure

- <!doctype html> tells the browser to use modern HTML.
- <html lang="en"> is the root element and declares the document language.
- <head> stores metadata, links to stylesheets, and the title.
- <body> contains the content displayed on the page.

Attributes and nesting

- Write attributes in the opening tag: Visit.
- Quote attribute values. Use meaningful values and avoid duplicate IDs.
- Nest elements correctly; close inner elements before outer elements.

Example

Practical code example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Structure Demo</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Common beginner mistakes

- Forgetting to close non-void elements.
- Putting visible page content in the head instead of the body.
- Using elements only for their default appearance rather than their meaning.

02. Text Formatting

Headings and paragraphs

- Use <h1> through <h6> for a meaningful heading hierarchy, not just to change font size.
- Use <p> for paragraphs. Browsers collapse most repeated spaces and line breaks in normal HTML text.
- Use
 only when a line break is part of the content, such as an address or poem.

Meaningful emphasis

- indicates strong importance; indicates emphasis.
- <b> and <i> are appropriate when the goal is attention or alternate voice without adding the same importance/emphasis meaning.
- <u> marks text with a non-textual annotation; do not use underline alone to make text look like a link.
- Use CSS for visual alignment and styling, for example text-align: center;.

Example

Practical code example

```
<h1>Main heading</h1>
<h2>Section heading</h2>
<p>This is a paragraph with <strong>important</strong> text and <em>emphasis</em>.</p>
<p>First line<br>Second line</p>
```

03. Links & Navigation

Anchor element

- The <a> element creates a hyperlink. The href attribute gives its destination.
- Use descriptive link text such as “Read the HTML guide” instead of “click here.”
- Absolute URL: full address such as https://example.com/about. Relative URL: path within your site such as about.html.

Internal links and page sections

- Use an element with an id, then link to it with href="#about".
- Navigation is commonly placed inside <nav>.
- When opening a new tab with target="_blank", use rel="noopener noreferrer".

Example

Practical code example

```
<nav aria-label="Main navigation">
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="#contact">Contact section</a>
</nav>

<section id="contact">
  <h2>Contact</h2>
  <p>Email: <a href="mailto:hello@example.com">hello@example.com</a></p>
</section>
```

04. Images & Multimedia

Images

- Use <img src="photo.jpg" alt="A description">. The src identifies the image and alt provides a text alternative.
- Write useful alt text for informative images; use alt="" for purely decorative images.
- Add width and height where possible to reserve space and reduce layout shift.

Video and audio

- Use native <video controls> and <audio controls> when appropriate.
- Provide captions for videos with speech; offer transcripts when useful.
- Use responsive CSS such as max-width: 100%; height: auto; for images.

Example

Practical code example

```
<img src="images/cat.jpg" alt="A cat sitting by a window"
     width="640" height="426">

<video controls width="640">
  <source src="media/demo.mp4" type="video/mp4">
  Your browser does not support video playback.
</video>

<audio controls>
  <source src="media/song.mp3" type="audio/mpeg">
  Your browser does not support audio playback.
</audio>
```

05. Lists & Tables

Lists

- is an unordered (bulleted) list; is an ordered (numbered) list; each item uses <li>.
- Use lists for grouped related items, navigation links, steps, and checklists.

Tables

- Use tables for genuinely tabular data, not for page layout.
- Use <table>, <caption>, <thead>, <tbody>, <tr>, <th>, and <td> appropriately.
- Use scope="col" or scope="row" on header cells to clarify relationships.

Example

Practical code example

```
<h2>Learning steps</h2>
<ol>
  <li>Learn HTML</li>
  <li>Practice CSS</li>
  <li>Build a project</li>
</ol>

<table>
  <caption>Course progress</caption>
  <thead>
    <tr><th scope="col">Topic</th><th scope="col">Status</th></tr>
  </thead>
  <tbody>
    <tr><td>HTML</td><td>Learning</td></tr>
    <tr><td>CSS</td><td>Next</td></tr>
  </tbody>
</table>
```

06. Forms & Input

Common form controls

- Place related controls in <form>. A <label for="email"> should match the control's id="email".
- Common types include text, email, password, number, date, checkbox, and radio.
- Use name so a control's value can be included when the form is submitted.
- Use required, suitable input types, and constraints for basic browser validation.

Submission and security

- The form's action identifies the destination and method commonly uses get or post.
- HTML validation improves usability but does not replace server-side validation.
- Never trust client-provided data; validate and sanitize it on the server. Do not put passwords or secrets in URLs.

Example

Practical code example

```
<form action="/contact" method="post">
  <label for="name">Name</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <label for="message">Message</label>
  <textarea id="message" name="message" rows="4" required></textarea>

  <button type="submit">Send message</button>
</form>
```

Useful reminder

- A submit button inside a form submits by default; use type="button" for a button that should not submit.

07. Semantic HTML5

What semantic HTML means

- Semantic elements describe the role of their content. They improve maintainability, accessibility, and how tools interpret the page.
- <header>: introductory content for a page or section.
- <nav>: major navigation links.
- <main>: the page's primary content; usually one visible main region per page.
- <section>: a thematic grouping, usually with a heading.
- <article>: self-contained content that could stand on its own.
- <aside>: related or complementary content.
- <footer>: footer for a page or section.
- <figure> and <figcaption>: a referenced illustration/media item and its caption.

Example

Practical code example

```
<header><h1>My Website</h1></header>
<nav aria-label="Main navigation"><a href="/">Home</a></nav>
<main>
  <article>
    <h2>News story</h2>
    <p>Article content goes here.</p>
  </article>
  <aside><p>Related links</p></aside>
</main>
<footer><p>&copy; 2026 My Website</p></footer>
```

08. Meta Tags & SEO

Metadata basics

- The <title> appears in browser tabs and is often used as the search result title.
- A meta description summarizes a page; search engines may use it in snippets, but can choose other text.
- <meta name="viewport" content="width=device-width, initial-scale=1.0"> supports expected mobile viewport sizing.
- Open Graph metadata can control how a page is previewed when shared on social platforms.
- Use a descriptive title and page-specific description; do not stuff keywords or promise guaranteed rankings.

Example

Practical code example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Beginner HTML Guide | Example Site</title>
  <meta name="description" content="Learn HTML structure, links, forms, and semantic markup.">
  <meta property="og:title" content="Beginner HTML Guide">
  <meta property="og:description" content="A practical introduction to HTML.">
  <meta property="og:type" content="website">
</head>
<body><h1>Beginner HTML Guide</h1></body>
</html>
```

09. Accessibility

Make content usable by more people

- Provide meaningful alt text for informative images.
- Connect each form control to a visible label.
- Ensure interactive features can be used with a keyboard and have a visible focus indicator.
- Use headings in a logical order and link text that explains its destination.
- Use native HTML controls first. ARIA can add missing semantics but cannot fix poor interaction or incorrect HTML.
- Maintain sufficient color contrast and do not rely on color alone to convey meaning.
- Test with keyboard navigation and, where possible, a screen reader.

ARIA examples

- Use aria-label only when a visible label or accessible name is otherwise missing or insufficient.
- Use aria-expanded on controls that open and close expandable content, keeping its value synchronized with the state.
- Do not add ARIA roles that conflict with native element semantics.

Example

Practical code example

```
<button type="button" aria-expanded="false" aria-controls="details">
  Show details
</button>
<div id="details" hidden>
  <p>Extra information appears here.</p>
</div>

<img src="chart.png" alt="Bar chart showing monthly sales rising from January to March">
```

Quick accessibility checklist

- Can you reach every interactive control using Tab and activate it using the keyboard?
- Does every meaningful image have a useful text alternative?

10. HTML Entities

Why entities are useful

- Entities let you write reserved characters and symbols in text safely.
- < displays <; > displays >; & displays &; " displays a double quote.
-   inserts a non-breaking space, but should not be used to create page layout.
- Unicode characters can also be written directly when the document uses UTF-8.

Example

Practical code example

```
<p>5 &lt; 8 and 8 &gt; 5.</p>
<p>Tom &amp; Ana said &quot;Hello&quot;.</p>
<p>Copyright &copy; 2026</p>
<p>Heart: &#9829; | Check: &#10003;</p>
```

11. Responsive Design

Responsive HTML foundations

- Set the viewport meta tag so mobile browsers use the device width.
- Use flexible images: img { max-width: 100%; height: auto; }.
- Prefer fluid layouts and content that can wrap; avoid fixed widths that force horizontal scrolling.
- Media queries are CSS, not HTML. They apply styles based on conditions such as viewport width.
- Use a mobile-first approach: write the base styles for small screens, then add enhancements for wider screens.

Example CSS

Practical code example

```
<!-- Put this inside the <head> -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Example CSS (in a stylesheet) -->
img {
  max-width: 100%;
  height: auto;
}

.container {
  width: min(100% - 2rem, 70rem);
  margin-inline: auto;
}

@media (min-width: 48rem) {
  .cards {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 1rem;
  }
}
```

Quick responsive checklist

- Check at narrow phone width, tablet width, and desktop width.
- Look for horizontal scrolling, clipped text, tiny buttons, and images overflowing their containers.

12. Best Practices

Clean, reliable markup

- Use lowercase element and attribute names and quote attribute values.
- Indent nested elements consistently and use meaningful file/folder names.
- Use one unique id per element; use class for reusable styling groups.
- Keep HTML for structure, CSS for presentation, and JavaScript for behavior.
- Remove obsolete presentational tags and inline styling when CSS is more suitable.
- Comment only where a note helps explain a non-obvious decision; keep comments current.
- Validate markup, test on different screen sizes, and check links, forms, images, and keyboard use.

Example

Practical code example

```
<!-- Clear structure, meaningful names, and valid nesting -->
<main class="page-content">
  <section aria-labelledby="intro-title">
    <h1 id="intro-title">Welcome</h1>
    <p>Keep markup readable and meaningful.</p>
  </section>
</main>
```

Quick reference: common HTML elements

Purpose	Elements
Document	html, head, body, title, meta
Text	h1–h6, p, strong, em, br
Navigation	a, nav
Media	img, video, audio, source, figure, figcaption
Grouping	div, span, main, section, article, header, footer, aside
Lists	ul, ol, li
Tables	table, caption, thead, tbody, tr, th, td
Forms	form, label, input, textarea, select, option, button

Practice exercises

- 1. Create a personal profile page with one h1, two sections, and a footer.
- 2. Add three navigation links: one to another page, one to an external site, and one to a section on the same page.
- 3. Add an image with useful alt text and make it responsive with CSS.
- 4. Build a numbered list of five steps and a table with at least three rows.
- 5. Create a contact form with name, email, message, labels, and required validation.
- 6. Replace generic divs with suitable semantic elements where their meaning is clear.
- 7. Add a title, meta description, viewport, and Open Graph title to a page.
- 8. Navigate your page using only the keyboard and fix any inaccessible controls.

- 9. Use entities to display <, >, &, and a copyright symbol.
- 10. Test your page at phone, tablet, and desktop widths.

Knowledge check

1. What is the difference between a tag and an element?

A tag is the markup token; an element includes the tags and the content between them, where applicable.

2. Which attribute supplies a link destination?

href.

3. Why is alt text used?

It provides a text alternative for an image, including for people using screen readers or when the image cannot load.

4. When should you use a table?

For data with row-and-column relationships, not for general page layout.

5. Does required replace server-side validation?

No. Server-side validation is still necessary.

6. Name three semantic elements.

Examples: main, nav, article, section, header, footer, aside.

7. What does the viewport meta tag do?

It helps the browser size the layout viewport appropriately on mobile devices.

8. What is the difference between id and class?

An id should uniquely identify one element in a document; a class can be reused on multiple elements.

Suggested learning order

1) Build a simple page → 2) headings and paragraphs → 3) links and images → 4) lists and tables → 5) forms → 6) semantic structure → 7) accessibility and SEO → 8) responsive CSS → 9) build a small project and validate it.

Next step: after HTML, learn CSS fundamentals (selectors, box model, Flexbox, Grid, responsive breakpoints), then JavaScript basics and DOM interaction.